LAST NAME (please print)	[redacted]
First name (please print)	[redacted]
Student Number	[redacted]

[redacted]
DEPARTMENT OF COMPUTER SCIENCE

[redacted]
– Final Exam –

Friday, Dec. 17, 2021, 2:00 - 5:00pm
[redacted]

Instructor: [redacted]

The exam will be available Friday at 2:00pm in [redacted] and you must submit your solutions by 5:10pm as a single pdf file. Students with accommodation for extra time will submit their solutions by email to [redacted], according to their accommodation details.

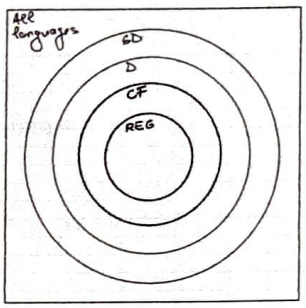
This exam consists of 5 questions (16 pages, including this page), worth a total of 100 marks. All answers are to be written in this booklet. The exam is 180 minutes long and comprises 39% of your final grade.

(1) 20pt	
(2) 20pt	
(3) 20pt	
(4) 20pt	
(5) 20pt	
Grade	

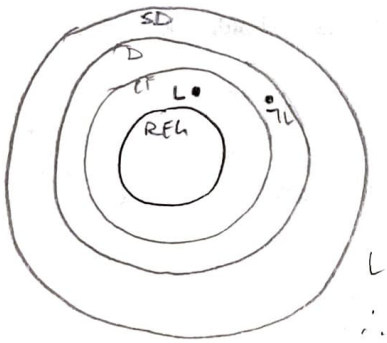
1. (20pt) Consider the context-free language of balanced parentheses of three kinds:

$$L = \{w \in \{(,), [,], \{,\}\}^* \mid \text{all parentheses in } w \text{ are properly balanced}\}.$$

Place $\neg L$ correctly in the correct area of the map. Prove your answer.



ALL LANGS



$A^n B^n C^n \leftarrow \neg$

NOT CONTEX-FREE

WOULD BASICALLY BE

$L = \{(^n,)^n, [^m,]^m, \{^p, \}^p\}$

$\therefore \neg L$ WOULD MAKE THEM UNBALANCED

THE LANGUAGE L IS NOT A REGULAR LANGUAGE BUT IS CONSIDERED A CONTEXT-FREE LANGUAGE BECAUSE AN UNAMBIGUOUS CONTEXT-FREE GRAMMAR CAN BE PRODUCED FOR EACH OF THE 3 SETS OF BRACKETS, WHICH CAN IN TURN BE COMBINED INTO ONE LARGE CONTEXT-FREE GRAMMAR

EX. GRAMMAR FOR ONE PAIR OF BRACKETS:

$S^* \to \varepsilon$
$S^* \to S$
$S \to SS_i$
$S \to S_i$
$S_i \to (S)$
$S_i \to ()$

} CAN BE APPLIED USING THE OTHER BRACKETS

- BECAUSE A CFG CAN BE CREATED FOR L, THEN L IS CONTEXT-FREE, BUT ITS COMPLEMENT WOULD NOT BE CONTEXT-FREE BECAUSE CONTEXT-FREE LANGUAGES ARE NOT CLOSED UNDER COMPLEMENT AND $\therefore \neg L$ WOULD BE EITHER CONTEXT-FREE OR NOT (COULD BE ANYTHING OUTSIDE THE CONTEXT FREE RANGE.
- A PDA (PUSH DOWN AUTOMATA) CAN BE CONSTRUCTED SUCH THAT IF THE NUMBER OF PARANTHESES COMES OUT UNBALANCED (STACK IS EMPTY, INPUT IS NOT OR INPUT IS EMPTY AND STACK IS NOT) THEN IT WILL ACCEPT WHICH WOULD MAKE IT CONTEXT-FREE AS THE CLASS OF LANGUAGES ACCEPTED BY PDA'S IS EXACTLY THE CLASS OF CONTEXT-FREE LANGS OR THE $\neg L$ COULD FAIL THE CONTEXT-FREE PUMPING THEOREM, MEANING THAT IF TWO SECTIONS OF THE STRING ARE PUMPED AND ITS UNBALANCED CONDITION DOES NOT HOLD, THEN IT WOULD NOT BE CONTEXT-FREE
- $\therefore \neg L$ CANNOT BE PLACED IN ONE SE POSITION

2. (20pt) Answer the following questions with True or False; prove your answers:

   (a) (2pt) Any regular language is context-free.

   (b) (2pt) Some context-free languages are regular.

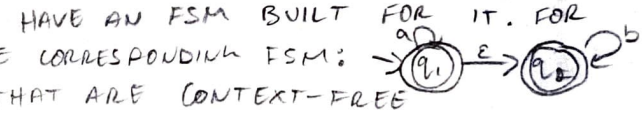
   (c) (8pt) Any regular language can be generated by an unambiguous context-free grammar.

   (d) (6pt) Any context-free grammar generating a regular language is unambiguous.

   (e) (2pt) For any non-semidecidable language L, we have that $\varepsilon \in L^*$.
   $\varepsilon \in L \in \neg SD$

a.) TRUE. ALL REGULAR LANGUAGES ARE A PROPER SUBSET OF CONTEXT-FREE LANGUAGES (WOULD MEAN THAT a^*b^* IS ALSO A CFL) SUCH AS $L=\{a^*b^*\}$ WHICH WHEN INTERSECTED WITH A CONTEXT-FREE LANGUAGE SUCH AS $L_2=a^nb^n$ $L_1 \cap L_2$, YOU GET A CONTEXT-FREE LANGUAGE. THIS LAW WOULD APPLY FOR ALL REGULAR LANGUAGES SUCH THAT ALL REGULAR LANGUAGES ARE CONTEXT-FREE

b.) TRUE. SOME CONTEXT-FREE LANGUAGES ARE REGULAR. SOME CONTEXT-FREE LANGUAGES CAN BE REGULAR WHEN LOOKING AT LANGUAGES THAT CAN HAVE AN FSM BUILT FOR IT. FOR EXAMPLE A LANGUAGE SUCH AS a^*b^* CAN HAVE THE CORRESPONDING FSM:



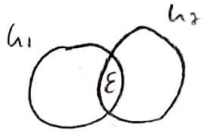
AND IS THEN CONSIDERED REGULAR. BUT LANGUAGES THAT ARE CONTEXT-FREE SUCH AS a^nb^n ARE ONLY CONTEXT-FREE NON-REGULAR AS SEEN BY THE INTERSECTION OF $(a^*b^*) \cap (a^nb^n)$ WHICH PRODUCES a^nb^n, A CONTEXT-FREE LANG

c.) TRUE. AN UNAMBIGUOUS CONTEXT-FREE GRAMMAR CAN PRODUCE BOTH AMBIGUOUS AND UNAMBIGUOUS GRAMMARS. THERE ARE SOME CONTEXT-FREE LANGUAGES THAT ARE INHERENTLY AMBIGUOUS, HOWEVER REGULAR LANGUAGES ARE PRODUCED/GENERATED BY REGULAR GRAMMARS WHICH HAVE STRICT CONDITIONS MAKING EVERY TERMINAL RIGHT-ASSOCIATIVE. IN TURN, THIS FORCED RIGHT ASSOCIATIVITY WOULD MEAN THAT ONLY ONE PARSE TREE CAN BE CREATED FOR EACH INPUT OF A REGULAR LANGUAGE DUE TO THE FORCED RIGHT BRANCHING. THIS WOULD MEAN THAT EVERY REGULAR LANGUAGE CAN BE GENERATED BY ONE PASE TREE MEANING THAT IT CAN BE GENERATED BY ONE/AN UNAMBIGUOUS CONTEXT-FREE GRAMMAR

(THE RULES INCLUDE RHS MUST HAVE ε, A SINGLE TERMINAL, OR A SINGLE TERMINAL FOLLOWED BY A SINGLE NONTERMINAL)

d.) FALSE. BECAUSE A CONTEXT-FREE GRAMMAR CAN BE AMBIGUOUS, UNAMBIGUOUS, OR INHERENTLY AMBIGUOUS, THAT MEANS THAT IF A CONTEXT-FREE GRAMMAR PRODUCES A REGULAR LANGUAGE, IT CAN BE PRODUCED BY EITHER OF THE TYPES OF GRAMMARS, SOME AMBIGUOUS GRAMMARS CAN BE REDUCED TO UNAMBIGUOUS, BUT SOME ARE STRICTLY AMBIGUOUS

e.) $L \in \neg SD$ IAN $\varepsilon \in L^*$ IS FALSE. NOT TOO SURE WHY BUT I GOT A GUT FEELING :P



3. (20pt) Consider the following language:

$$L = \{<G_1, G_2> \mid G_1, G_2 \text{ context-free grammars such that } L(G_1) \cap L(G_2) = \{\varepsilon\}\}\ .$$

Here is an algorithm that decides L:

1. **if** $\varepsilon \notin L(G_1)$, **then reject** } REJECTED
2. **if** $\varepsilon \notin L(G_2)$, **then reject** }
3. $k_1 \leftarrow$ the k-constant in pumping theorem for G_1 } K - CONST
4. $k_2 \leftarrow$ the k-constant in pumping theorem for G_2 }
5. $k \leftarrow \max(k_1, k_2)$
6. **for all** $w \in L(G_1) - \{\varepsilon\}$ with $|w| \leq k$ **do**
7. **if** $w \in L(G_2)$ **then reject**
8. **for all** $w \in L(G_2) - \{\varepsilon\}$ with $|w| \leq k$ **do**
9. **if** $w \in L(G_1)$ **then reject**
10. **accept**

CAN'T DECIDE SOMETHING LIKE THIS

The main idea of the algorithm is that we can check membership for finitely many strings. Therefore, the difficulty arises in the case when both languages are infinite. We know from pumping theorem that, if a language has strings longer than k, then it has also strings shorter than k (we "pump out" to reduce the length, while staying in the language). The algorithm uses this idea with k the larger of the two individual k's of each language, to ensure the strings longer than k are long enough for both languages.

Is the above algorithm correct? Explain your answer.

THE ALGORITHM ABOVE IS NOT CORRECT. L WOULD BE UNDECIDABLE, LOOKING AT A REDUCTION FROM A KNOW LANGUAGE THAT IS NOT IN D, SUCH AS THE POST CORRESPONDENCE PROBLEM. $PCP \leq L$

$PCP = \{<P> : P \text{ HAS A SOLUTION}\}$

$\downarrow R$

$L = \{<G_1, G_2> : L(G_1) \cap L(G_2) = \{\varepsilon\}\}$

SUCH THAT THE REDUCTION MACHINE PCP WOULD CONSTRUCT BOTH THE GRAMMARS AND IF ORACLE EXISTED THEN THEN THERE WOULD HAVE TO BE ONE SOLUTION SUCH THAT G_1 AND G_2 GENERATE ε, WHICH IT WOULD ACCEPT, OTHERWISE IF NO SOLUTION IS FOUND THE OVERALL MACHINE WOULD REJECT. THE MACHINE $C = \neg Oracle(R(<P>))$

Oracle → $R(<P>)$ = • CONSTRUCT G_1 AND G_2 } IF $<P> \in PCP$: THERE IS ONE SOLUTION SO BOTH G_1 AND G_2 WILL GENERATE ε. Oracle rejects. C ACCEPTS.
• RETURN $<G_1, G_2>$

IF $<P> \notin PCP$: THERE IS NO SOLUTION SO ORACLE ACCEPTS. AND C REJECTS

↳ BUT THERE IS NO MACHINE THAT CAN DECIDE PCP SO NEITHER ORACLE NOR L CAN BE IN D.

4. (20pt) Consider an alphabet Σ such that all languages below are over Σ. For each of the languages below, prove, without using Rice's theorem, whether it is in D, SD − D, or ¬SD.

   (a) $L_1 = \{<M> |$ M Turing machine, the complement of $L(M)$ is semidecidable$\}$. $\neg L(M) \in SD$

   (b) $L_2 = \{<M> |$ M nondeterministic finite automaton such that, for any nondeterministic finite automaton M' with $L(M') = L(M)$, M has the same number of states as M' or fewer$\}$. → D
   (For L_2, any finite automaton M can be encoded as $< M >$ using any of the methods we studied, for instance, the one for Turing Machines. The way the encoding is done is irrelevant for the question.)

   (c) $L_3 = \{<M> |$ M Turing machine, both $L(M)$ and $\neg L(M)$ are infinite$\}$. SD − D

   (d) $L_4 = \{<M, w> |$ on input w, M begins by moving right one square onto w, then it never moves outside w, i.e., outside the $|w|$ tape cells occupied by w at the beginning$\}$. → D

   (All Turing machines are single-tape deterministic.)

a.) $L_1 = \{<M> |$ M TM. $\neg L(M) \in SD\}$ IS NOT IN SD ($\neg$SD)

L_1 CAN BE REDUCED FROM A LANGUAGE SUCH AS THE NON-HALTING LANGUAGE, $\neg H$ WHERE IF Oracle (AS SHOWN BELOW) EXISTS, THEN IT SEMIDECIDES $\neg H$ AND THUS SEMIDECIDES L_1. $\neg H \leq L_1$

(Oracle:) $R(<M,w>)$ = MAKE M# SUCH THAT:
1.1. IT SAVES INPUT X,
1.2. ERASES TAPE
1.3. WRITES w ON TAPE
1.4. RUNS M ON w
1.5. WRITES X TO TAPE
1.6. IF VALID $<M', w'>$, RUN M' ON w'
1.7. IF NOT ↑, REJECT
1.8. ACCEPT

$<M,w> \in \neg H$: $\neg L(M)$ WOULD BE AN ELEMENT OF SD, MEANING Oracle WOULD ACCEPT

$<M,w> \notin \neg H$: IF M' HALTS THEN Oracle DOES NOT ACCEPT

- HOWEVER NO MACHINE EXISTS TO SEMIDECIDE $\neg H$, SO Oracle CANNOT EXIST AND ∴ L_1 IS $\neg$SD

b.) L_2 WOULD BE IN D BECAUSE A TURING MACHINE CAN BE CONSTRUCTED TO DECIDE IT. THIS TM M_X WOULD NOTE HOW MANY STATES THERE ARE FOR M' AND WOULD RUN M SUCH THAT IF IT HAS THE SAME NUMBER OR FEWER STATES AS M' THEN ACCEPT, BUT IF IT HAS MORE STATES THAN M', REJECT.

c.) L_3 WOULD BE IN $\neg$SD. L_3 CAN BE REDUCED BY THE NON-HALTING PROBLEM. IF YOU ASSUME L IS INFINITE AND ITS COMPLEMENT ARE INFINITE THEN WHEN YOU RUN THE INFINITE LANGUAGE THROUGH Oracle BELOW, IF IT IS SEMIDECIDABLE THEN SO IS $\neg H$.

Oracle $R(<M,w>)$: CONSTRUCT $M\#(x)$
1.1 SAVES x (ALREADY A CONTRADICTION BECAUSE NOT SINGLE-TAPED
1.2 ERASE TAPE
1.3 WRITE w
1.4 RUN M ON w FOR $|x|$ STEPS/UNTIL IT HALTS
1.5 IF HALTED, LOOP
1.6 ELSE ACCEPT ← LAST STEP FOR NEXT PART
2. RUN SAME ON $\neg L(M)$
3. RETURN $(<M,H>)$

$<M,w> \in \neg H$. M DOESN'T HALT ON EITHER L OR $\neg L$, ACCEPTS EVERYTHING. Oracle ACCEPTS

$<M,w> \notin \neg H$. M HALTS AND LOOPS AND Oracle DOES NOT ACCEPT

- HOWEVER NO MACHINE EXISTS TO SEMIDECIDE $\neg H$, SO Oracle CANNOT EXIST AND ∴ L_1 IS $\neg$SD

EXISTS AN INPUT WHERE TM HALTS IN FEWER THAN |w| STEPS

d.) L_4 IS IN D. A TURING MACHINE CAN BE CREATED TO DECIDE L_4 SUCH THAT, AFTER MOVING RIGHT ONE SQUARE INTO W IT CAN SEE IF THE MACHINE HALTS WITHIN |w| STEPS, MOVING RIGHT ON THE TAPE. IF IT DOES HALT WITHIN |w| STEPS, ACCEPT, ELSE (IF IT GOES MORE THAN |w| STEPS (BY ONE)) REJECT. (>) BECAUSE A TURING MACHINE CAN BE CREATED TO DECIDE L_4 THEN L_4 IS IN D

5. (20pt) Recall that TILES = $\{< T >|$ any finite surface on the plane can be tiled with the tile set $T\}$? (Recall that tiles cannot be rotated or flipped and adjacent sides of joined tiles must have the same colour.)

   (a) (6pt) Is the set of tiles below in TILES? Explain your answer.

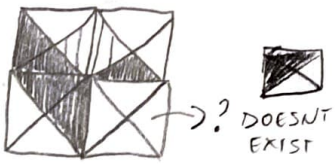
B → BOTTOM, TOP LEFT, RIGHT
WHITE → BOTTOM, TOP LEFT RIGHT

   (b) (6pt) If your answer above is negative, then is it possible to add a single tile that makes the new set (of four tiles) in TILES? Explain your answer.

   (c) (2pt) Which of TILES and ¬TILES is harder? No proof is required.
   For this question, "harder" means (strictly) higher in the hierarchy: $D < SD - D < \neg SD$, that is, $\neg SD$ is harder than $SD - D$, which is harder than D; also, no class is harder than itself.

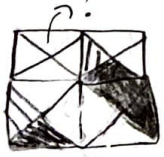
   (d) (6pt) Given a language L such that $\neg L$ is harder than L, is it possible that L is in: (i) D, (ii) $SD - D$, (iii) $\neg SD$? Explain your answer. You can use examples studied in class without proof.

a.) THE SET OF TILES ABOVE IS NOT IN TILES. IF YOU TAKE THE FIRST SQUARE IT WOULD REQUIRE A GREY TRIANGLE ON THE LEFT TO MATCH THE GREY IT HAS ON THE RIGHT, WHICH CAN BE FOUND IN THE 3rd TILE. THEN THE SAME 1st TILE WOULD NEED A BLACK TRIANGLE ON THE TOP TO MATCH THE ONE ON THE BOTTOM WHICH CAN BE FOUND IN THE 2nd TILE. HOWEVER THE 2nd TILE COMBINED W/ THE 3rd TILE WOULD NEED A TILE THAT HAS BLACK ON THE TOP AS WELL AS THE LEFT, WHICH DOES NOT EXIST AND ∴ THIS SET OF TILES IS NOT IN TILES (DEPICTED BELOW)



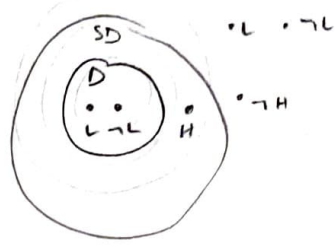
b.) NO IT WOULD NOT BE POSSIBLE TO ADD A SINGLE TILE THAT MAKES THE NEW SET IN TILES. IF A TILE SUCH AS  WERE ADDED, THEN IT FACES AN ISSUE OF THERE BEING NO TILE W/ A WHITE TRIANGLE ON THE LEFT SIDE TO MATCH ITS WHITE RIGHT SIDE. SO IT WOULD HAVE TO BE  . HOWEVER, ANOTHER ISSUE WILL BE RUN INTO WHEN TILE 1 IS JOINED WITH TILE 3 ON ITS RIGHT, AND THE NEW TILE, (TILE 4). ON TOP OF ERRORS WITH COMBINATIONS OF THE NEW TILE, THE ORIGINAL 3 TILES HAVE MORE THAN ONE ADJACENT SIDE ISSUES ∴ NO ONE TILE ADDITION WILL BE ABLE TO SOLVE THIS ISSUE, MAKING THE NEW SET INTO TILES

c.) EX.



C →

FIG 1.



C.) NEITHER TILES NOR ¬TILES CAN BE CONSIDERED "HARDER" (I HATE THEM BOTH EQUALLY). TILES HAS BEEN PROVEN TO BE ¬SD AND THEREFORE, THAT WOULD MEAN THAT ITS COMPLEMENT, ¬TILES WOULD HAVE TO ALSO BE ¬SD DUE TO THE LAWS OF UNDECIDABLE LANGUAGES. ∴ BECAUSE THEY ARE PART OF THE SAME CLASS, NEITHER CAN BE HARDER THAN THE OTHER

d.) IF ¬L IS HARDER THAN L, THEN

i.) IT IS NOT POSSIBLE TO BE IN D BECAUSE IF L WERE IN D, THEN SO WOULD ¬L, BASED ON THE LAWS IN FIG 1. FOR EXAMPLE $L = \emptyset$, THEN ¬L WOULD BE Σ^* AND BOTH ARE DECIDABLE AND THUS NOT HARDER THAN THE OTHER

ii.) IT IS POSSIBLE TO BE SD-D BECAUSE IF L WERE IN SD THEN ITS COMPLEMENT WOULD BE ¬SD AND THUS HARDER. GIVEN THAT H, THE HALTING LANG IS SD-D, ITS COMPLEMENT CANNOT REPLACE THE ACCEPT W/ LOOP AND THUS ¬L WOULD BE ¬SD

iii.) NOT POSSIBLE TO BE IN ¬SD. BY THE SAME LOGIC AS i.) IF L WERE IN ¬SD THEN ¬L WOULD BE SD.